



ARCHITECTURE & SYSTEM DESIGN

Sidekick

A local-first personal AI assistant for a developer's whole working day.

One private memory of your work feeds five capabilities and several workflows, delivered through three thin surfaces. Everything runs on your machine — nothing leaves it unless you explicitly opt in.

37

HTTP ENDPOINTS

88

AUTOMATED TESTS

0

NPM DEPENDENCIES

1

EVENT CONTRACT

AES-256

GCM AT REST

● Local-first

● Propose, don't act

● Provenance on everything

● Quiet by default

● Inspectable & reversible

Contents

A single, accurate reference to the whole product — what it is, the one contract behind everything, how the pieces fit, and how each workflow moves.

01	What Sidekick is	2	06	External connectors	13
02	The canonical data model	3	07	The three surfaces	14
03	System architecture	4	08	Security & data lifecycle	14
04	Capabilities & workflows	6	09	Honest limitations	16
05	Workflows in motion	7			

01 What Sidekick is

Sidekick keeps **one private, on-device record** of everything you do across a working day and turns it into help you can trust. That single memory powers five capabilities and several workflows, reached through three thin surfaces. Nothing leaves your machine by default.

● The companion

A single Node.js process, bound only to loopback (127.0.0.1:4317), with bearer-token auth on every request and an encrypted SQLite store (AES-256-GCM at rest). It ships with **zero npm dependencies**.

● Where it lives

Everything sits under `~/.sidekick/` — the SQLite database, the salt, a startup canary, keychain-fallback secrets, and your `TODD.md`. The whole footprint is one folder you own.

Principles — enforced in code, not just intent

● Local-first

Capture, storage, and derivation all happen on-device. The network is opt-in, never assumed.

● Propose, don't act

Sidekick surfaces suggestions — commitments, todos, minutes — and waits for you to confirm or dismiss.

● Provenance on everything

Every derived item carries where it came from and why it fired, so nothing is a black box.

● Quiet by default · inspectable & reversible

It stays ambient unless something is urgent, and every record can be inspected, edited, or deleted.

02 The canonical data model

One contract behind everything.

Every surface and every connector sends the **same shape** — a `WorkContextEvent` — to `POST /events`. This single contract is *why* one shared brain is possible: surfaces and connectors stay thin, while the companion alone owns capture, storage, derivation, and policy.

WorkContextEvent fields

id unique event id	ts timestamp	source origin surface (1 of 10)
kind event type (1 of 9)	ref link / identifier	summary human-readable text
project project grouping	confidence derivation certainty	origin work or personal

FIELD	ALLOWED VALUES
source · 10	editor · github · jira · slack · teams · mail · calendar · meeting · docs · browser
kind · 9	edited_file · opened_file · reviewing_pr · opened_ticket · mentioned · message · read_doc · meeting_action · calendar_event
origin · 2	work · personal

What the companion derives from each event

From every event, up to three record types are derived and stored in SQLite tables:

TABLE	WHAT IT HOLDS
<code>events</code>	The append-only timeline (logical delete — nothing is truly erased mid-flow).
<code>commitments</code>	Promises owed by me / owed to me. Status moves proposed → confirmed / dismissed .
<code>lessons</code>	Durable insights — the "learned..." and "root cause..." moments worth keeping.
<code>memory_records</code>	Working memory (clears at day end) or semantic memory (kept).
<code>focus_sessions</code>	Deep-work timers.
<code>audit_log</code>	Every mutation, recorded.

03 System architecture

Thin surfaces in · one companion in the middle · opt-in services out.

Requests enter through a single authenticated HTTP API. Events are normalized and redacted, derived into records, and written to encrypted storage; capabilities and workflows read back from that same store. AI enrichment and external connectors are optional and clearly bounded — nothing crosses the machine boundary unless you turn it on.

COLOUR LANGUAGE

Surface	thin client	Companion core	internal module	Optional AI	enrichment
Encrypted store	SQLite / DB	Markdown file	you own it	Secrets / keychain	key material
External · read-only	GitHub / Graph	Cloud LLM · opt-in	Anthropic	Local LLM	Ollama
Decision	branch point	Stop / delete	no-op · blocked	Surfaced output	to the user

The full system map is on the next page (landscape), using the colour language above.

03 System architecture — map

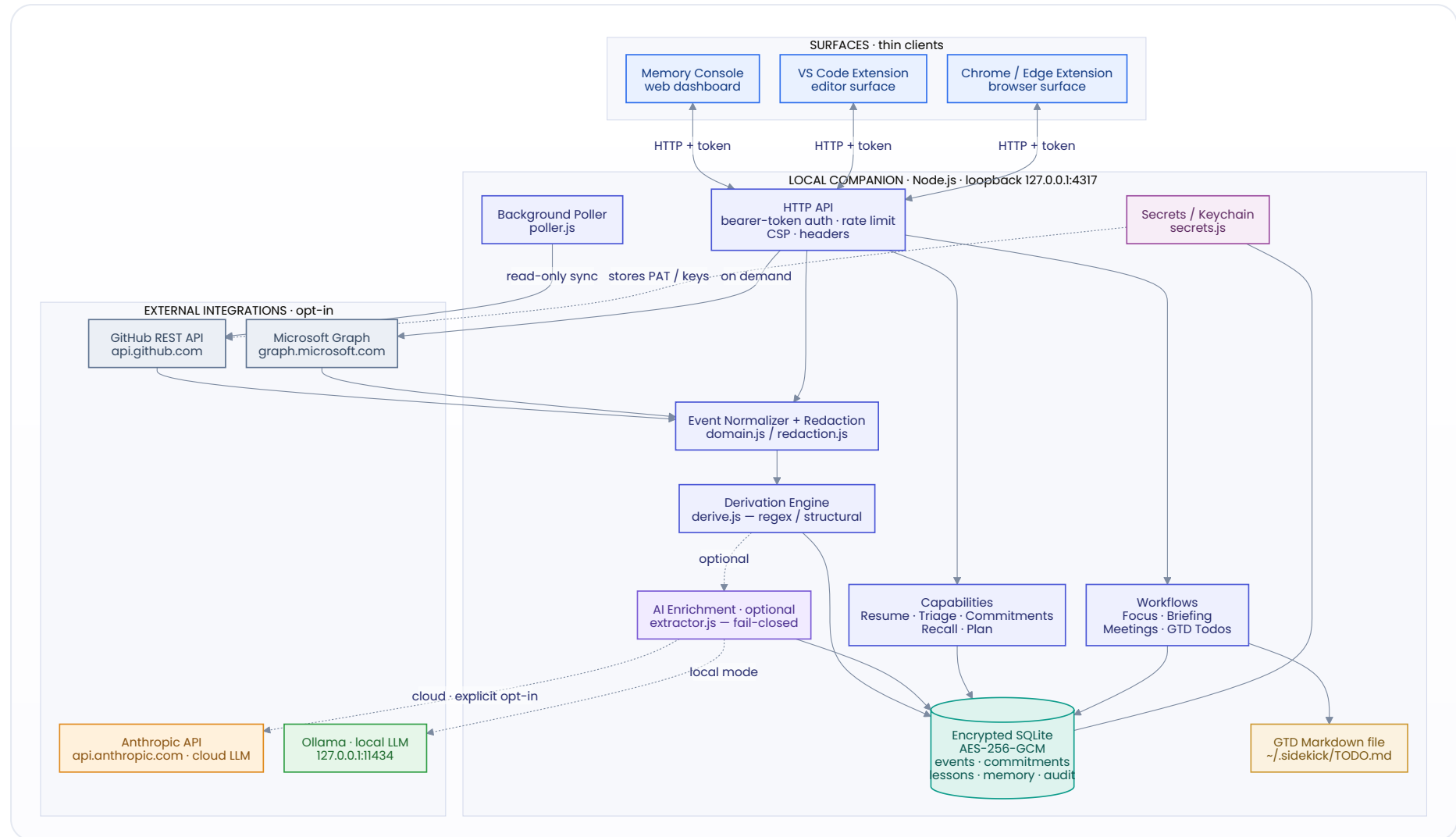


Figure 3 · The companion sits between thin surfaces and opt-in external services. Solid arrows are the default local path; dashed arrows are optional or cross-boundary.

04 Capabilities & workflows

Five capabilities answer a question about your day; workflows string them into routines.

Capabilities

CAPABILITY	THE QUESTION IT ANSWERS	ENDPOINT
 Resume	"Where was I?" — reconstructs context before the last interruption gap.	GET /resume
 Triage	"What needs me now?" — ranked by urgency × importance.	GET /triage
 Commitments	"What did I promise?" — owed-by-me / owed-to-me, with confirm & dismiss.	GET /commitments · POST /commitments/status
 Recall	"What did I learn?" — keyword search over lessons and notes.	GET /recall?q=
 Plan	"What's realistic today?" — a proposed shape for the day.	GET /plan

Workflows

WORKFLOW	WHAT IT DOES	ENDPOINTS
 Focus Session	A protected deep-work timer; ambient except for urgent items, with a summary on completion.	POST /focus/start · GET /focus/current · POST /focus/complete · GET /focus/sessions
 Briefing	A proactive digest — Resume + Triage + Commitments + open todos.	GET /briefing
 Meeting lifecycle	Prep → ingest notes/transcript → decisions & action items become tracked commitments → minutes.	GET /meeting/prep · POST /meeting/ingest · GET /meeting/minutes
 GTD Todos	A plain-Markdown task board: Inbox / Today / Next / Waiting / Someday / Done.	GET /todos · POST /todos/{add,check,move,reschedule,remove,rollover,sync}
 Inspect / Privacy	Timeline, audit log, and delete-source-history (which cascades to derived records).	GET /events · GET /audit · DELETE /events commitments lessons memory

05 Workflows in motion

Six flows that show how activity becomes trustworthy, reversible help.

5.1 • The heartbeat — capture → derive → surface

The core loop. An activity is captured by a surface or connector, posted once to /events, normalized and redacted, appended to the encrypted timeline, then derived into commitments, lessons, and memory. Capabilities read the result and surface it **with provenance**.

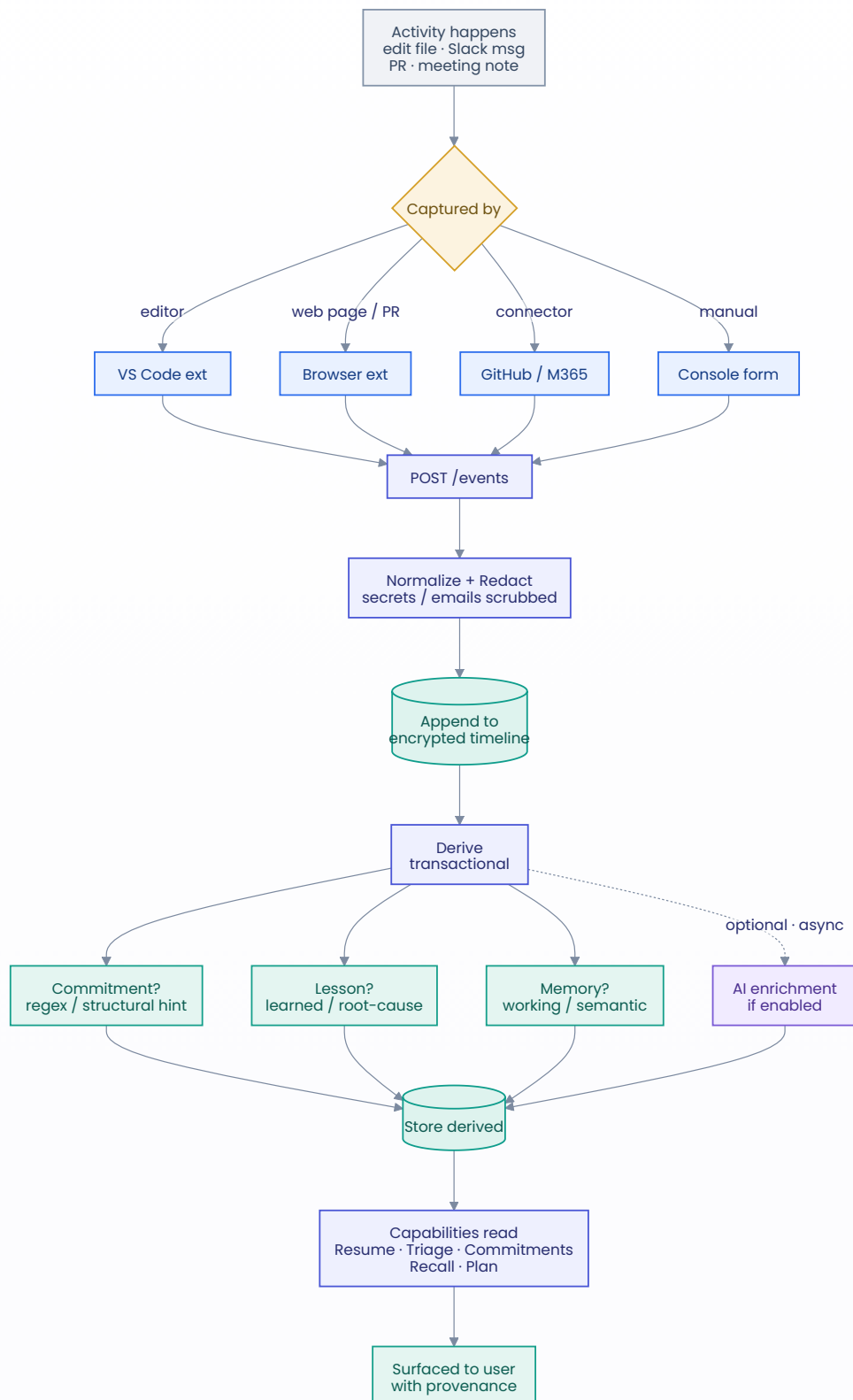


Figure 5.1 • One capture path, one contract, many derivations.

5.2 · Commitments lifecycle — propose → confirm / dismiss

A commitment is only ever *proposed*. Structural hints (a PR review request, a meeting action) set direction explicitly; regex hints ("can you...", "I will...") infer it; negations ("just FYI", "ignore") produce nothing. You decide — and a confirmed commitment **survives** re-derivation.

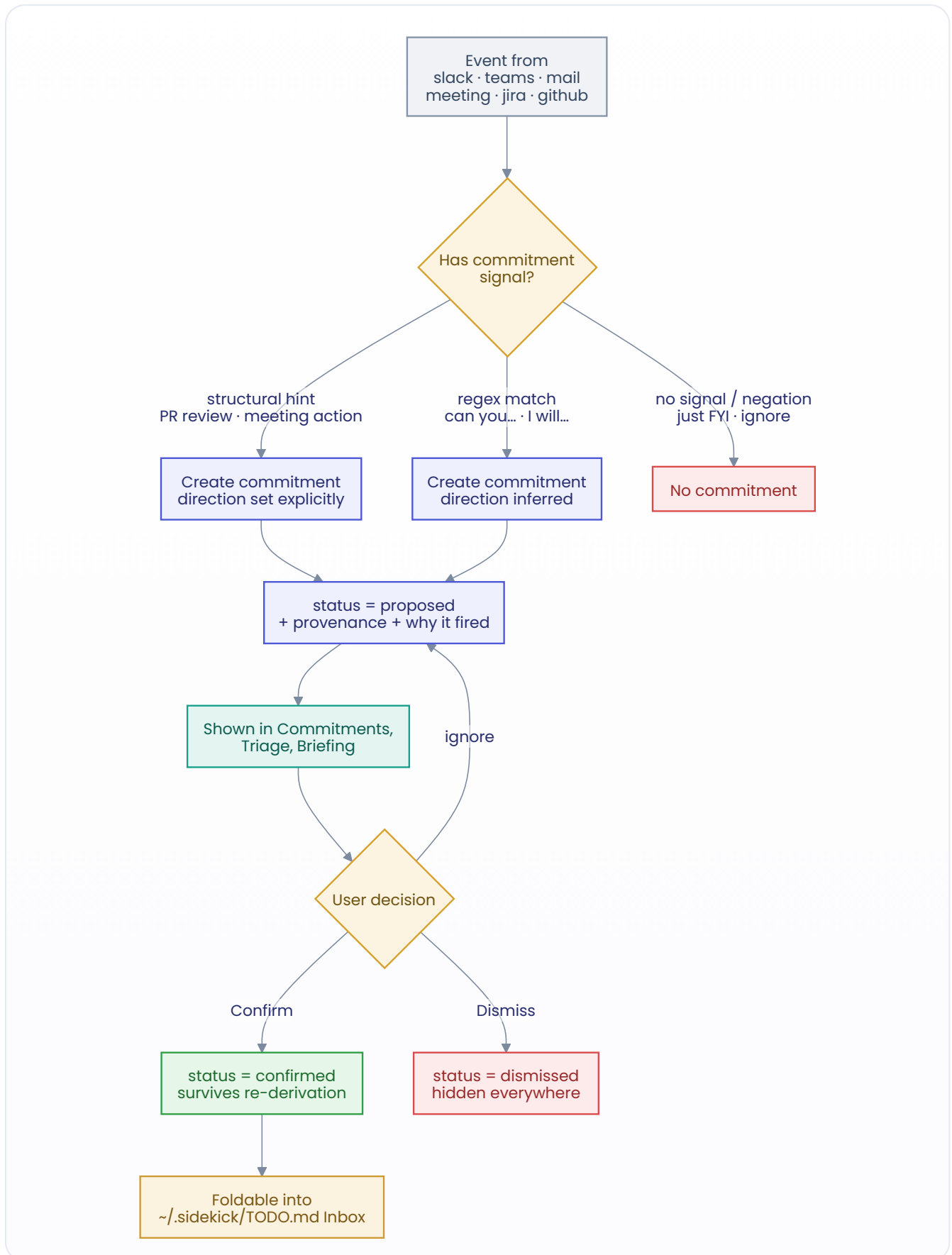


Figure 5.2 · Nothing is tracked without a signal, and nothing is acted on without you.

5.3 · Meeting lifecycle — prep → minutes → tracked follow-ups

Before a meeting, Sidekick surfaces the relevant past. During it, notes or a transcript are ingested; decisions and owned action items become **tracked commitments** with provenance back to the meeting. Afterwards, minutes read the decisions and actions back.

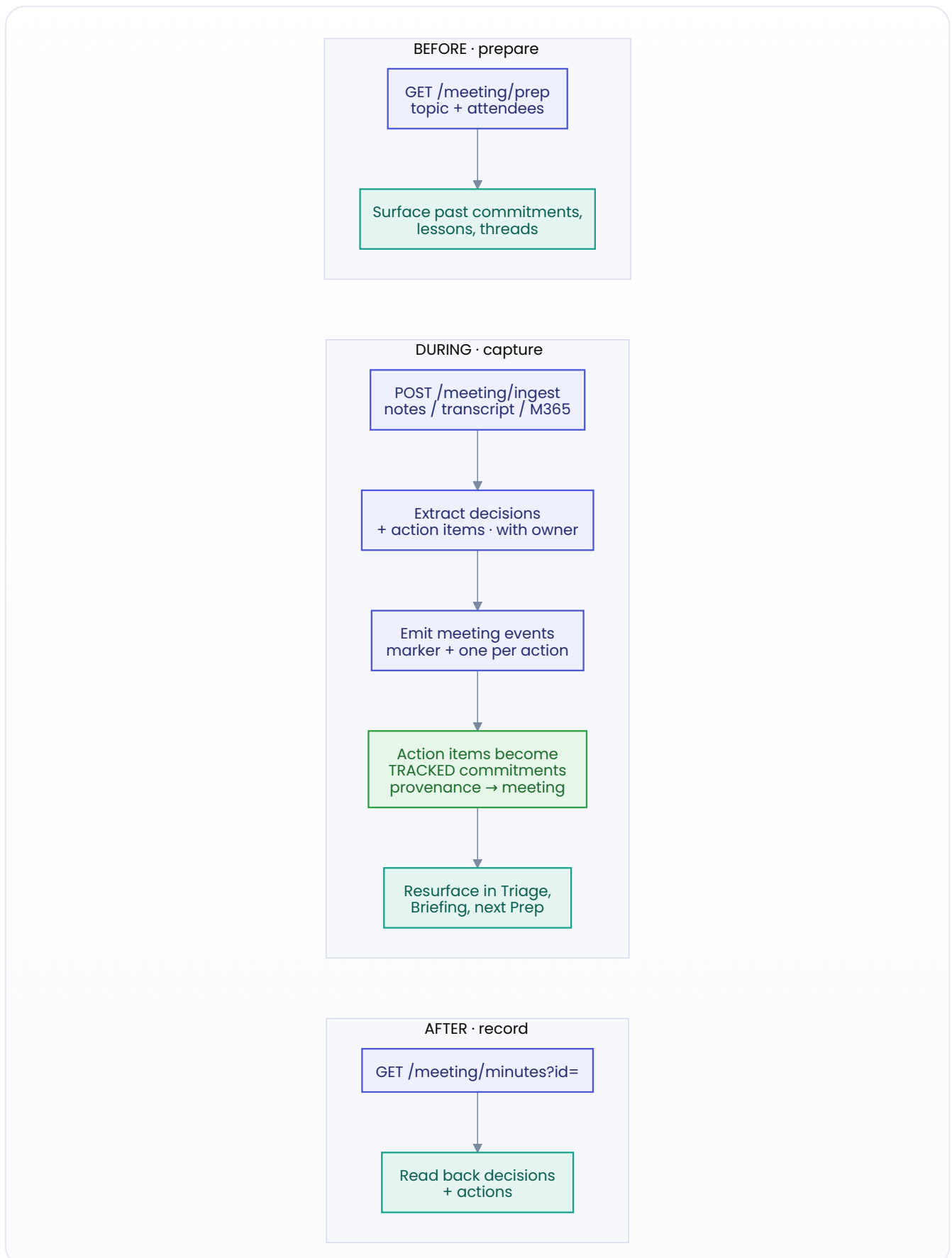


Figure 5.3 · Three phases — before, during, after — that leave a durable trail.

5.4 GTD todos — plain Markdown, propose-don't-act

The task board is a Markdown file you own (~/.sidekick/TODO.md). Captured commitments are appended (de-duped) to the Inbox; you clarify, move, reschedule, check, roll over, or remove. Sidekick never silently rewrites your list.

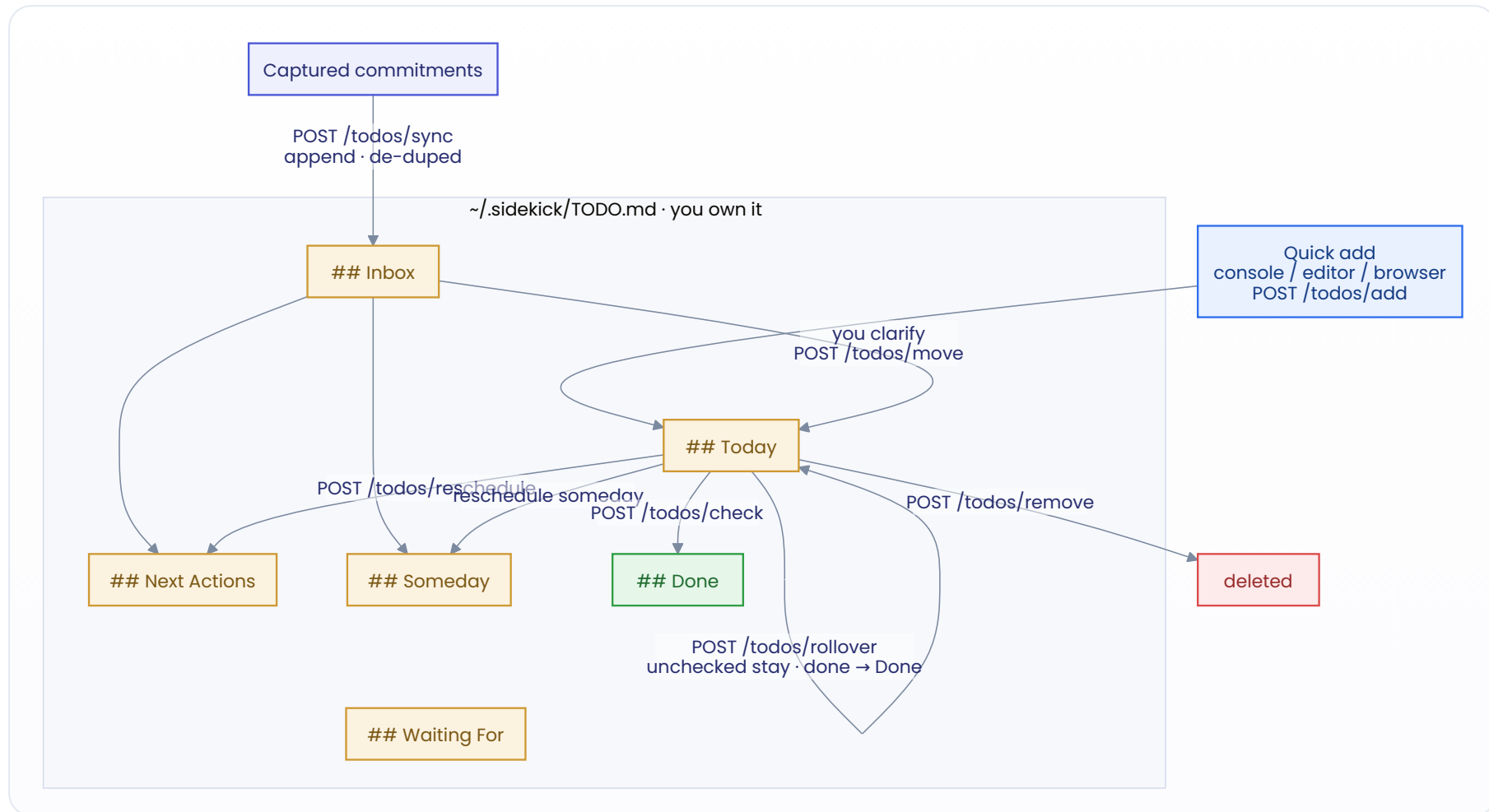


Figure 5.4 · Inbox → Today → Next / Waiting / Someday → Done, all in plain text.

5.5 · AI extraction tiers — fail-closed on egress

Extraction has a floor and a ceiling. The **regex floor** always works offline. Above it, `SIDEKICK_LLM_MODE` chooses local, cloud, or auto. The rule that matters: **auto never selects cloud**, even with an API key present — accidental egress is blocked by design, and any cloud payload is source-gated and redacted first.

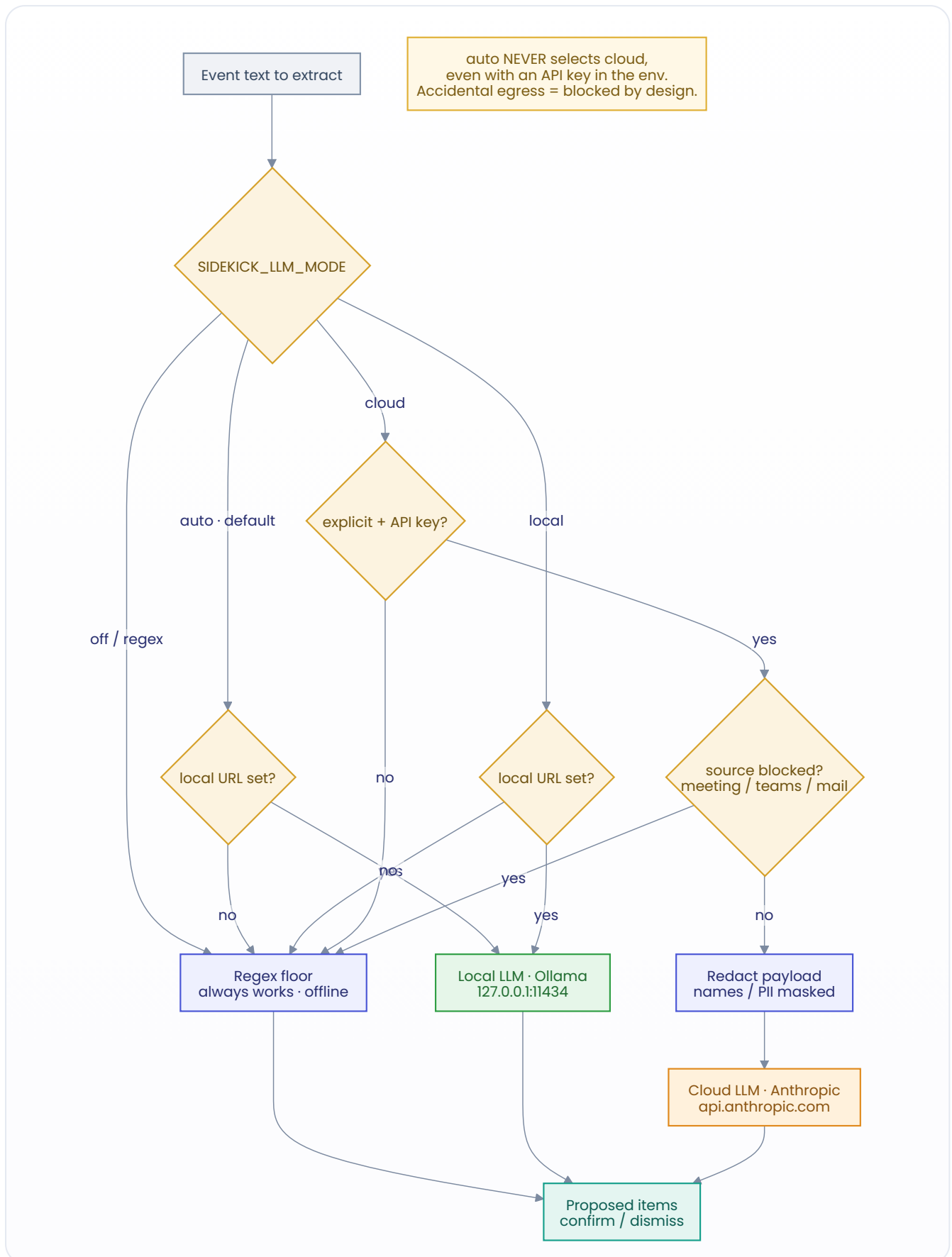


Figure 5.5 · Local by default, cloud only on explicit, gated, redacted opt-in.

5.6 · Background GitHub polling

On a timer (default 5 minutes), if a GitHub PAT is stored, Sidekick performs a **read-only** sync: review requests become owed-by-me, assigned issues/PRs become owned-by-me. Each becomes a `WorkContextEvent` and flows through the same `normalize → derive → store` path. No PAT means a silent no-op.

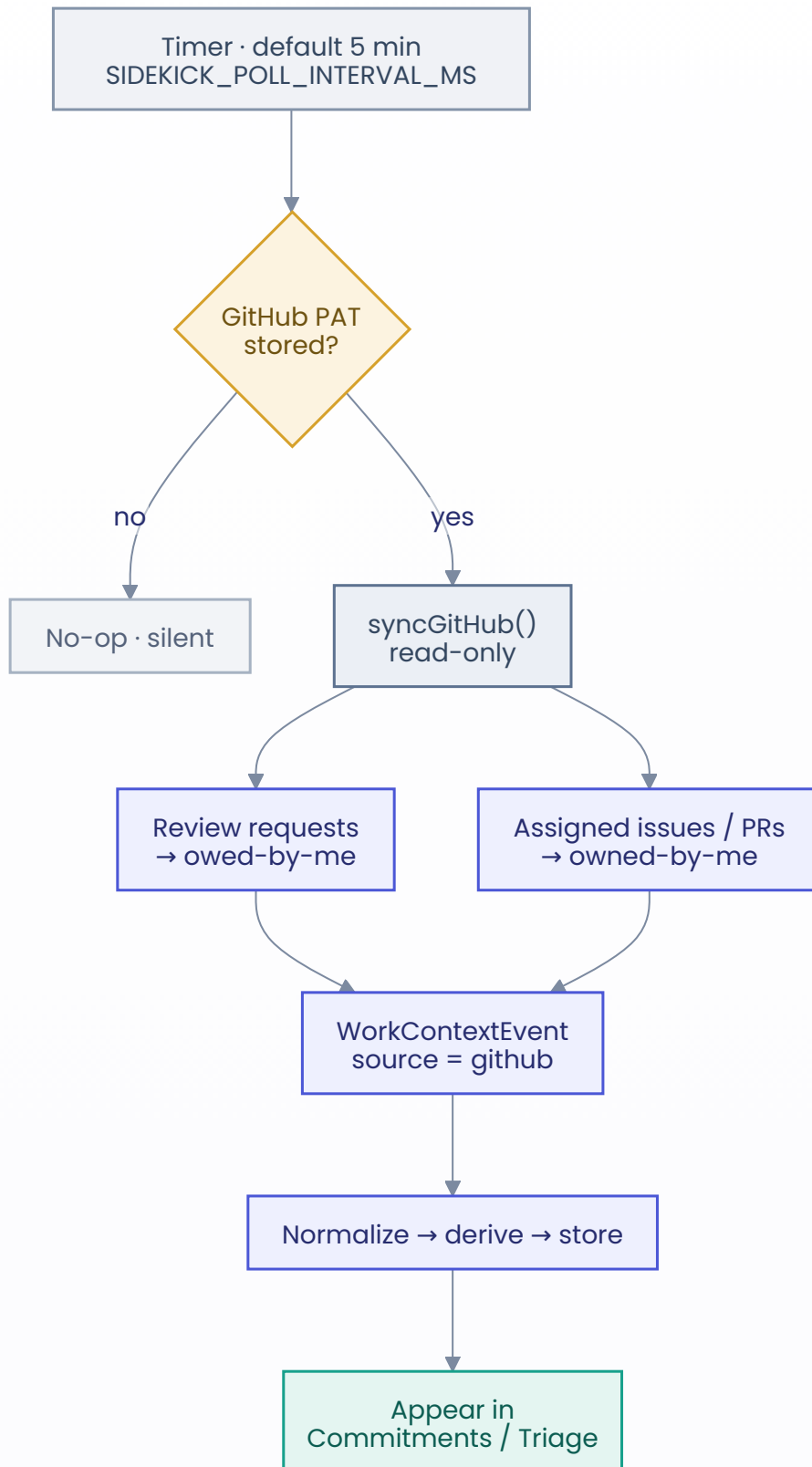


Figure 5.6 · A quiet, read-only pull that feeds the same pipeline.

06 External connectors & integrations

All external calls are opt-in and read-only. Nothing leaves the machine by default.

INTEGRATION	ENDPOINTS	AUTH	DIRECTION	NOTES
GitHub	api.github.com – /user, /search/issues	Fine-grained PAT (no OAuth), in OS keychain	Read-only, pull	Polled in background (default 5 min) or on demand. Routes: <code>POST /connectors/github/{token, sync}</code> , <code>GET /connectors/github/status</code> .
Microsoft 365 (Graph)	graph.microsoft.com/v1.0 – /me/events, /me/messages, /chats/{id}/messages, /me/onlineMeetings + /transcripts	Graph access token (pasted, not stored)	Read-only, pull	On demand: <code>POST /connectors/microsoft/sync</code> . Covers calendar, mail, Teams, and meeting transcripts.
Anthropic API (cloud LLM)	api.anthropic.com/v1/messages · default claude-haiku-4-5	API key (explicit opt-in only)	Send redacted text	Never auto-selected; source-gated; payload redacted. AI <i>enriches</i> , it is never required.
Ollama (local LLM)	127.0.0.1:11434/api/generate · default llama3.1	none (localhost)	Local only	Privacy-mode extraction – nothing leaves the machine.
OS Keychain	macOS security / Linux secret- tool (libsecret)	–	Local secret store	Stores the API token + GitHub PAT, with a 0600-file fallback.

Capture inputs — how events get in

editor (VS Code ext) · browser (Chrome ext, infers github/jira from URL) · github (connector) · plus slack / teams / mail /
calendar / meeting / docs / jira via Microsoft Graph · manual capture (console form) · or anything POSTing to /events.

Not yet integrated — honest gaps

Native OAuth for Slack, Jira / Linear, and GitLab is not built. These can post to /events, but there is no built-in connector for them yet.

07 The three surfaces

Each surface is thin — capture and display only. The companion holds the intelligence.

SURFACE	ROLE	TALKS TO
Memory Console 127.0.0.1:4317/	Home + admin: briefing, all capabilities, task board, timeline, connectors, and privacy controls.	Companion HTTP API (auto-loads token)
VS Code Extension	Ambient capture (<code>opened_file</code> / <code>edited_file</code>); a panel with capabilities, tasks, and focus; quick-add.	Companion HTTP API (token in SecretStorage)
Chrome / Edge Extension	Capture the current page / PR / ticket; quick add-to-today; focus status.	Companion HTTP API (token in extension storage)

08 Security & data lifecycle

The foundation everything else stands on.

Key guarantees

- Loopback-only bind · timing-safe token auth · per-client rate limiting.
- Encryption key **decoupled** from the API token — rotating the token can't brick the database.
- Strict CSP nonce on the dashboard · transactional writes · **fail-closed** AI egress.
- Cascade delete of derived records · a complete audit trail of every mutation.

08 Security & data lifecycle — flow

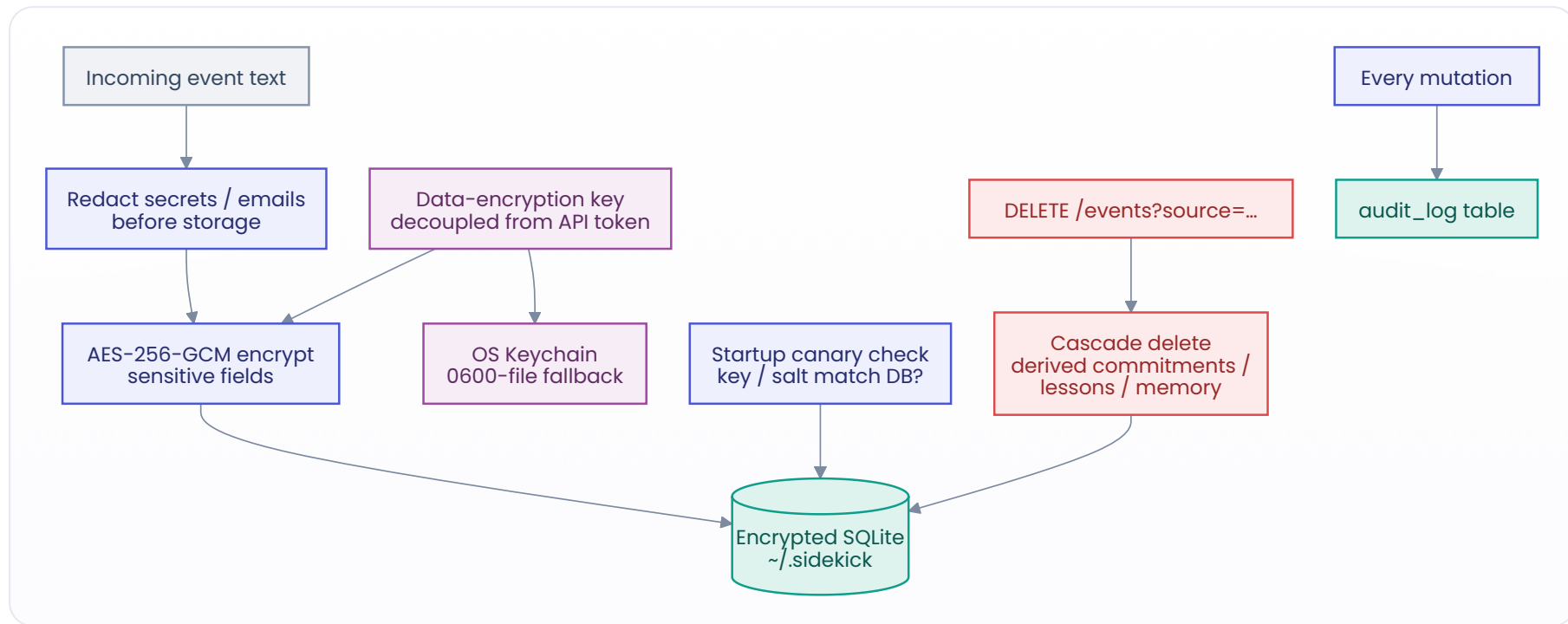


Figure 8 · Redact → encrypt → store, with keys decoupled from the API token, a startup canary, cascade deletes, and a full audit trail.

09 Honest limitations

Stated plainly — the trade-offs and gaps that come with the design.

- **Breadth is a deliberate bet** — covering the whole working day is the riskiest scoping choice.
- **AI extraction** is wired and unit-tested with mock models, but **not yet benchmarked against a live LLM**.
- **Recall** is keyword search, not vector embeddings (and is labeled as such).
- **Connectors** cover GitHub + Microsoft 365 only; Slack / Jira / GitLab still need auth.
- `node:sqlite` is Node's experimental SQLite, guarded at startup.

Source of truth: the code under `sidekick/companion/src/` — 37 HTTP endpoints, 88 tests. This document reflects the implementation, not a spec written from memory.